

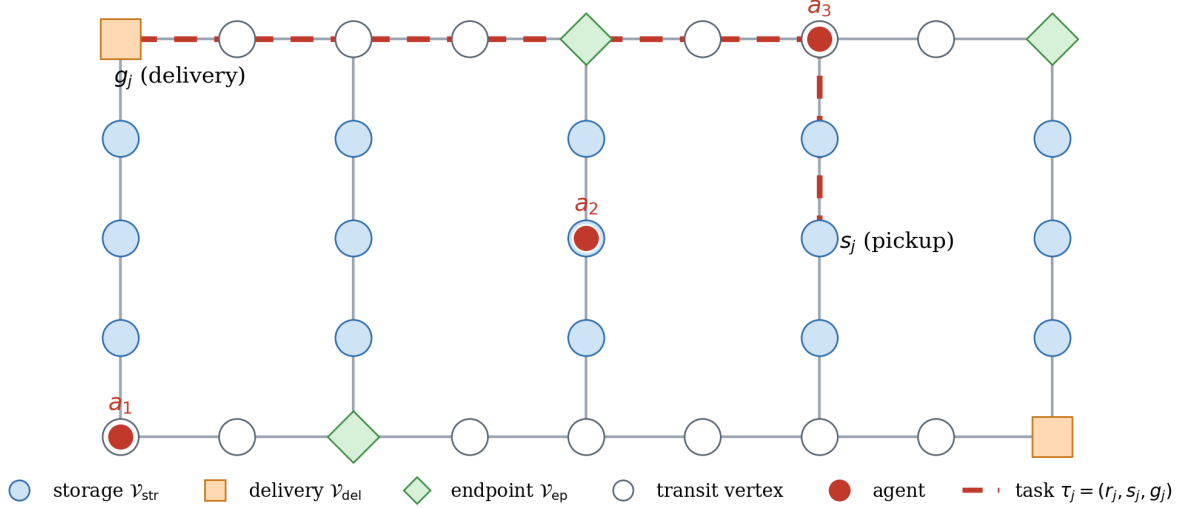


Fig. 1. A small warehouse instance. Vertex shape encodes role: storage faces, delivery points, endpoints and plain transit vertices. A single task $\tau_j = (r_j, s_j, g_j)$ is shown as a route from its pickup vertex s_j to its delivery vertex g_j . The layout is not a lattice: aisle structure, not grid geometry, determines which movements exist.

I. PROBLEM FORMULATION

This section defines the coupled storage–routing system studied in this thesis. It proceeds in the order in which a simulation run unfolds: the environment, the agents that move in it, the storage state, the work that arrives, the controllers that decide, what a decentralised agent can see, the resulting learning problem, the loop that closes between the two layers, and the quantities used to score a run. Each object is described in plain terms before it is written formally.

Throughout, this section states *what* the objects are, not *how* they are computed. Realisations — heuristics, algorithms, network architectures and hyperparameters — are deferred to Section ??, and software concerns to Section ??.

A. The environment

The warehouse is a finite graph $G = (V, E)$. Vertices are the discrete positions a robot can occupy: aisle cells, cross-aisle junctions, storage faces and drop-off points. Edges are the movements that are physically possible between two adjacent positions. A rectangular grid is one special case, but the formulation deliberately does not assume a lattice, because real warehouses contain one-way aisles, diagonal shortcuts and irregular blocks.

Four roles partition the vertices by what can happen at them. $V_{\text{mov}} \subseteq V$ are the vertices an agent may occupy. $V_{\text{str}} \subseteq V$ are storage faces, where items are held and from which pickups are made. $V_{\text{del}} \subseteq V$ are delivery or drop-off points, where completed tasks terminate. $V_{\text{ep}} \subseteq V_{\text{mov}}$ are endpoints: vertices at which an idle agent may remain without blocking transit. Each edge $e \in E$ carries a traversal cost $c(e) > 0$, and $d_G(v, w)$ denotes the shortest-path cost from v to w .

Figure 1 shows a small instance with all four roles marked.

At each timestep an agent at vertex v either waits or traverses one outgoing edge. Writing $N^+(v) = \{w \in V_{\text{mov}} : (v, w) \in E\}$ for the movable successors of v , the action set at v

is

$$\mathcal{U}(v) = \{\text{wait}\} \cup \{\text{move}(w) : w \in N^+(v)\}. \quad (1)$$

Waiting costs $c_{\text{wait}} \geq 0$ and moving from v to w costs $c(v, w)$, so the realised one-step cost is

$$\hat{c}(v, w) = \begin{cases} c_{\text{wait}}, & v = w, \\ c(v, w), & (v, w) \in E. \end{cases} \quad (2)$$

The two cases are needed because a self-loop (v, v) is not an element of E , so c is undefined on a wait and any route cost containing a wait would otherwise be ill-typed.¹

Some baseline algorithms are complete only on *well-formed* instances. Where such a baseline is used, the following is imposed as an assumption on the generated instance rather than derived: at least m non-task endpoints exist, and between any pair of endpoints there is a path that does not pass through a third endpoint. Instances are checked against this condition in Section ??, and the condition is reported only for the baselines to which it applies.

B. Agents and collisions

A fleet $\mathcal{A} = \{a_1, \dots, a_m\}$ of homogeneous agents moves on G . Agent a_i occupies $\ell_i(t) \in V_{\text{mov}}$ at time t and selects $u_i(t) \in \mathcal{U}(\ell_i(t))$. Its realised route over the horizon is $p_i = (\ell_i(0), \dots, \ell_i(T))$. A single agent's transition is legal when

$$\ell_i(t+1) = \ell_i(t) \quad \vee \quad (\ell_i(t), \ell_i(t+1)) \in E. \quad (3)$$

Legality of each agent's own move is not sufficient, because agents share the infrastructure. Two agents collide if they end a timestep at the same vertex, or if they exchange positions across a single edge. A joint transition is therefore *collision-free* when both

$$\ell_i(t+1) \neq \ell_j(t+1) \quad \forall i \neq j, \quad (4)$$

$$\neg(\ell_i(t) = \ell_j(t+1) \wedge \ell_j(t) = \ell_i(t+1)) \quad \forall i \neq j. \quad (5)$$

The two conditions are logically independent. Constraint (4) inspects the successor state alone; (5) inspects the pair (state, successor) and is invisible to any check performed only at $t+1$. Omitting it permits two agents to pass through one another in a single-lane aisle. Figure 2 illustrates both, together with the feasible alternative in which one agent waits.

Constraints (4)–(5) are hard: a run in which they are violated is not a solution to this problem. They are stated here as feasibility conditions on the realised trace. A learned policy induces a distribution over joint actions whose support may contain infeasible ones, so feasibility cannot be delegated to a penalty term in a reward; the mechanism that enforces it is specified in Sections ?? and ?? and verified in Section ??.

C. Where the items are

Items are held at storage faces, and where they are held determines where pickups happen. Let $\mathcal{K}$ be the set of SKU types. The storage state must be able to represent split inventory, multiple units, and finite shelf space, so it is defined as a counting function

$$x_t : \mathcal{K} \times V_{\text{str}} \rightarrow \mathbb{N}_0,$$

¹The value of c_{wait} relative to edge costs determines whether an agent prefers to yield or to detour; it is therefore a parameter of the study rather than a constant of the model, and is reported in Section ??.

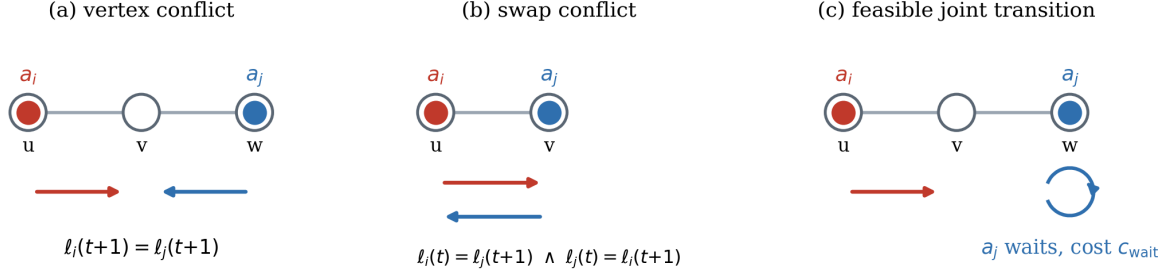


Fig. 2. The two forbidden joint transitions and one feasible alternative. Panel (c) shows the resolution the model always admits: waiting is a legal action at every vertex, so a collision-free joint action always exists. Which agent yields is a decision, not a constraint, and is fixed in Section ??.

where $x_t(k, v)$ is the number of units of SKU k held at storage vertex v at time t . Let $b_k > 0$ be the capacity consumed by one unit of SKU k and $\text{cap}(v) > 0$ the capacity of vertex v . The set of feasible configurations is

$$\mathcal{X} = \left\{ x \in \mathbb{N}_0^{|\mathcal{K}| \times |V_{\text{str}}|} : \sum_{k \in \mathcal{K}} b_k x(k, v) \leq \text{cap}(v) \quad \forall v \in V_{\text{str}} \right\}. \quad (6)$$

A simpler mapping $\sigma_t : \mathcal{K} \rightarrow V_{\text{str}}$ would force one location per SKU, unit quantity and unlimited shelf space; it is admissible only if those three restrictions are stated explicitly. Note that (6) is a system of $|V_{\text{str}}|$ knapsack constraints, so capacity couples the placement decisions of different SKUs at a shared vertex.

The storage layer acts on a slow clock. At epochs $t \in \{\Delta, 2\Delta, \dots\}$ the configuration is updated by a rule F ,

$$x_t = F(x_{t-\Delta}, \hat{\rho}_t, \hat{\mu}_t, \hat{w}_t) \in \mathcal{X}, \quad (7)$$

where $\hat{\rho}_t$ estimates per-SKU demand, $\hat{\mu}_t(e)$ estimates traversal counts per edge, and $\hat{w}_t(e)$ estimates waiting attributed to an edge or its downstream contention point. All three are computed only from data available strictly before the epoch, so (7) is causal. The fixed-storage baseline is the identity rule $F_{\text{fix}}(x, \cdot) = x$, defined directly rather than as the informal limit $\Delta = \infty$.

Because capacity couples SKUs, the update cannot be a per-item greedy score applied independently. It is a joint constrained decision over $\mathcal{X}$,

$$x_t \in \arg \min_{x \in \mathcal{X}} \left[\alpha D(x; \hat{\rho}_t) + \beta K(x; \hat{\mu}_t, \hat{w}_t) + \chi R(x, x_{t-\Delta}) \right], \quad (8)$$

where D measures demand-weighted access distance, K measures predicted congestion exposure, and R penalises reassignment. In words: D says popular items should be close, K says they should not all sit on the same corridor, and R says the warehouse should not be rearranged every epoch. The three functional forms, the weights α, β, χ , and the heuristic that actually searches $\mathcal{X}$ are specified in Section ??; (8) is an integer program and is not solved to optimality. Setting $\chi = 0$ removes relocation cost from the model entirely and is recorded in Section I-J as a limitation rather than a simplification in passing.

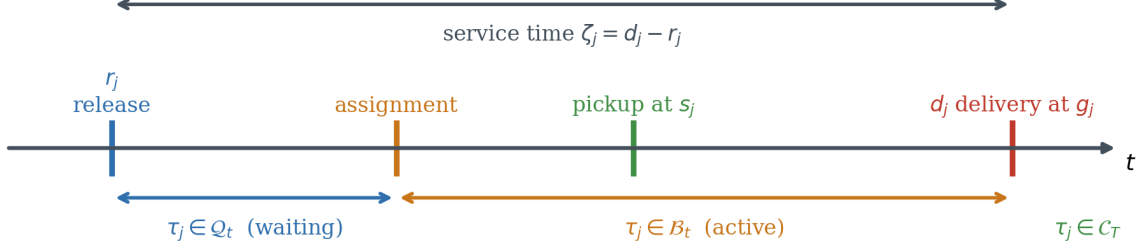


Fig. 3. Lifecycle of a task. The service time ζ_j spans both the waiting and the active phase, so a controller cannot improve it by delaying assignment. The membership sets $\mathcal{Q}_t$, $\mathcal{B}_t$ and $\mathcal{C}_t$ are disjoint at every timestep, a property asserted at runtime in Section ??.

D. Where the work comes from

Orders arrive continuously. At each timestep, orders are drawn from a distribution P_{ord} , an order being a finite multiset of SKU–quantity pairs, and each order is converted into one or more transport tasks by a task-generation rule. A task is a triple

$$\tau_j = (r_j, s_j, g_j), \quad r_j \in \mathbb{N}_0, \quad s_j \in V_{\text{str}}, \quad g_j \in V_{\text{del}}, \quad (9)$$

consisting of a release time, a pickup vertex and a delivery vertex. The expected number of tasks released per timestep is λ_{task} .

The pickup vertex is not free. For the requested SKU k_j it must satisfy

$$x_{r_j}(k_j, s_j) > 0. \quad (10)$$

Equation (10) is the formal link between the two layers and the central object of this thesis: it makes the spatial distribution of pickup vertices a function of the storage configuration, so any change to x_t changes the routing problem that the controller subsequently faces. If inventory depletion and replenishment are not simulated, x_t is interpreted as a placement template rather than a physical inventory level, and this interpretation is stated with the results.

A task passes through three disjoint sets over its life. It enters the waiting set $\mathcal{Q}_t$ at release, moves to the active set $\mathcal{B}_t$ when execution begins, and enters the completed set $\mathcal{C}_t$ on delivery. One convention for the boundary between $\mathcal{Q}_t$ and $\mathcal{B}_t$ is used throughout: execution begins at *assignment*, not at arrival of the agent at s_j . Writing d_j for the completion time, the service time is

$$\zeta_j = d_j - r_j, \quad (11)$$

which spans both phases and therefore cannot be improved by delaying assignment. Figure 3 shows the lifecycle and the interval that (11) measures.

E. Who decides

A controller $\pi \in \Pi$ performs two logically distinct jobs. *Task assignment* maps free agents and waiting tasks to agent–task pairs; *routing* maps the resulting agent targets and the available system information to actions $u_i(t) \in \mathcal{U}(\ell_i(t))$. Both jobs are treated as part of π , so that a path planner is not described as though it also solves assignment.

TABLE I. Information regimes. The three architectures are distinguished at the problem level only by what each component may condition on. Their algorithmic realisations are given in Section ??.

Architecture	Assignment conditions on	Routing conditions on
Centralised	global state	global state
Section-based	zone-local state, boundary protocol	zone-local state
Decentralised	global state (held fixed across arms)	$o_i(t)$ only

Controllers differ in *what information they may condition on*, and that is the only distinction the problem formulation needs. Three information regimes are studied and are summarised in Table I. A centralised controller may condition on the global state. A section-based controller partitions the movable vertices into zones,

$$\mathcal{Y} = \{Y_1, \dots, Y_p\}, \quad \bigcup_{q=1}^p Y_q = V_{\text{mov}}, \quad Y_q \cap Y_{q'} = \emptyset \quad (q \neq q'), \quad (12)$$

and may condition only on zone-local information, plus an explicit protocol at zone boundaries. A decentralised controller conditions only on the local observation defined in Section I-F and selects actions from a shared parameterised policy,

$$u_i(t) \sim \pi_\theta(\cdot \mid o_i(t)).$$

Parameter sharing is appropriate for homogeneous agents and does not make execution centralised; where global information is used during training only, the method is described as centralised training with decentralised execution.

Task assignment is held identical across the three architectures. Only routing differs. This is a design decision of the study rather than a property of the problem, and it is made so that a difference in results is attributable to the routing architecture and not confounded with assignment quality.

F. What a decentralised agent sees

A decentralised agent cannot see the warehouse. It sees a bounded neighbourhood of its own position, and it can exchange information with nearby agents. Both bounds are properties of the problem, because they define what a solution is permitted to depend on.

For observation depth d , the visible vertex set and the induced subgraph are

$$V_i^{(d)}(t) = \{v \in V_{\text{mov}} : d_G(\ell_i(t), v) \leq d\}, \quad \mathcal{G}_i^{(d)}(t) = G[V_i^{(d)}(t)]. \quad (13)$$

The size of (13) is bounded independently of $|V|$, which is what allows a single policy to be evaluated on warehouses larger than those it was trained on. The price is that anything beyond d hops is invisible, so an agent may commit to a move that becomes a conflict a few steps later.

Direction to a distant target is restored inside the local window by annotating each visible vertex with its distance to the agent’s current target $q_i(t)$,

$$\eta_i(v, t) = d_G(v, q_i(t)). \quad (14)$$

The annotation (14) is a potential: its difference along an edge, $\eta_i(w, t) - \eta_i(v, t)$, is negative exactly on shortest-path moves. Following the negative gradient is the shortest path, and congestion avoidance is precisely the choice to accept a non-negative step now

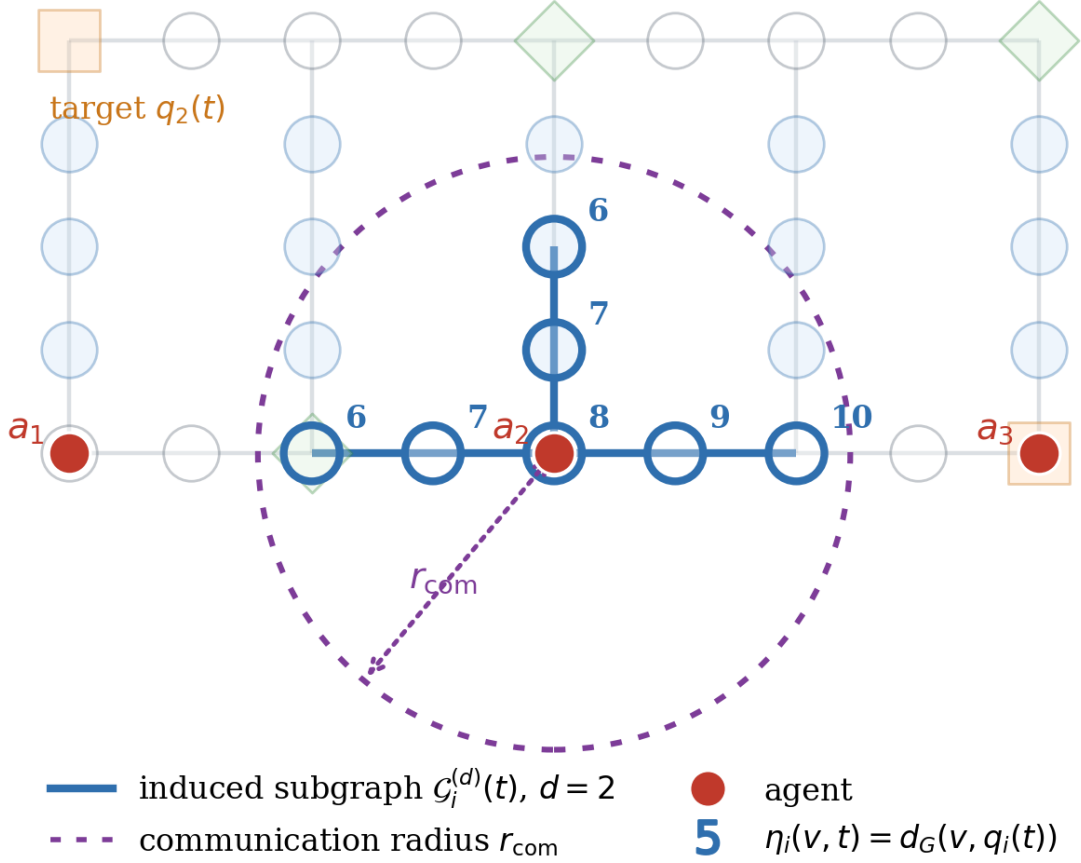


Fig. 4. The observation of a single agent. The depth- d induced subgraph is drawn at full strength and the rest of the warehouse is dimmed; numbers are the target distances $\eta_i(v, t)$. The dashed circle is the communication radius r_{com} . One agent lies outside both bounds and is therefore invisible, although it may cause a conflict within a few timesteps.

in exchange for a lower expected total later. Although d_G may be precomputed, η_i itself changes whenever the target changes, which happens at every reassignment.

Agents within communication range form a second, time-varying graph whose nodes are agents rather than locations,

$$G_t^{\mathcal{A}} = (\mathcal{A}, E_t^{\mathcal{A}}), \quad (a_i, a_j) \in E_t^{\mathcal{A}} \iff i \neq j \wedge d_G(\ell_i(t), \ell_j(t)) \leq r_{\text{com}}. \quad (15)$$

The environment graph G and the communication graph $G_t^{\mathcal{A}}$ have different node sets, different cardinalities and different edge semantics (“can move to” versus “can exchange information with”); they must not be conflated. The full observation available to agent a_i is

$$o_i(t) = \left(\mathcal{G}_i^{(d)}(t), \eta_i(\cdot, t), \text{messages from } \{a_j : (a_i, a_j) \in E_t^{\mathcal{A}}\} \right). \quad (16)$$

How $o_i(t)$ is encoded into a vector, and what a message contains, are modelling choices specified in Section ???. Figure 4 illustrates both bounds on one instance.

G. The learning problem

Because each agent acts from $o_i(t)$ while the environment evolves under the joint action of all agents, the decentralised setting is a partially observable stochastic game

$$\mathcal{M} = \left(\mathcal{A}, \mathcal{S}, \{\mathcal{U}_i\}_{i=1}^m, P, \{\mathcal{O}_i\}_{i=1}^m, \{O_i\}_{i=1}^m, \{R_i\}_{i=1}^m, \gamma \right), \quad (17)$$

where $\mathcal{S}$ is the state space, P the transition kernel, $\mathcal{O}_i$ the observation space, O_i the observation function, and $R_i : \mathcal{S} \times \mathcal{U}^m \times \mathcal{S} \rightarrow \mathbb{R}$ the reward function. The distinction between spaces and functions on the one hand and realised states, observations and rewards on the other is maintained throughout.

A local congestion feature, defined so that an agent does not count itself, is

$$\delta_i(t) = \frac{|\{a_j \in \mathcal{A} \setminus \{a_i\} : \ell_j(t) \in V_i^{(r_{\text{cng}})}(t)\}|}{\max\{1, |V_i^{(r_{\text{cng}})}(t)| - 1\}}. \quad (18)$$

Because the agents are homogeneous and cooperative and share one parameter vector, the learning objective is stated at team level,

$$\max_{\theta} J(\theta) = \max_{\theta} \mathbb{E}_{\pi_{\theta}} \left[\sum_{t=0}^{T-1} \gamma^t \frac{1}{m} \sum_{i=1}^m R_i(t) \right]. \quad (19)$$

The reward functions R_i are left unspecified here. A reward is an instrument for obtaining a policy that scores well on the system-level criterion of Section I-I; it can be redesigned without changing the problem, and it is therefore specified in Section ??.

H. How the two layers close the loop

Let $s_t \in \mathcal{S}$ contain the agent states, the task lifecycle sets and the current storage configuration. The system runs on two clocks. On every timestep, the fast dynamics advance under the controller,

$$s_{t+1} \sim P(\cdot \mid s_t, u_t), \quad u_t \sim \pi(\cdot \mid s_t),$$

with the corresponding product of local policies under decentralised execution. At storage epochs only, the slow dynamics additionally apply (7).

The loop closes in two directions. Forward, a change in x_t changes which vertices appear as pickups through the coupling condition (10), and therefore changes the routing problem. Backward, the realised routing statistics μ_t and w_t enter the next storage update through (7). Neither direction exists in a study that fixes one layer, and Figure 5 shows that the fixed-storage baseline is exactly this diagram with the backward path removed.

The separation of timescales has a consequence that constrains the study. Between epochs, the fast process is approximately stationary and the storage rule sees only its aggregate statistics. If Δ is too small relative to the mixing time of the fast process, F reacts to noise and the task distribution becomes non-stationary from the learner's point of view; if Δ is too large, the estimates $\hat{\rho}, \hat{\mu}, \hat{w}$ are stale. The choice of Δ is therefore an experimental variable, not a constant.

I. How a run is scored

A run is scored on completed work, on the cost of producing it, and on how the traffic was distributed while producing it. For the completed set $\mathcal{C}_T$ over a horizon T , throughput and mean service time are

$$\Lambda_T = \frac{|\mathcal{C}_T|}{T}, \quad \bar{\zeta}_T = \frac{1}{|\mathcal{C}_T|} \sum_{\tau_j \in \mathcal{C}_T} (d_j - r_j), \quad (20)$$

and the movement cost per completed task is

$$\bar{c}_T = \frac{\sum_{i=1}^m \sum_{t=0}^{T-1} \hat{c}(\ell_i(t), \ell_i(t+1))}{|\mathcal{C}_T|}. \quad (21)$$

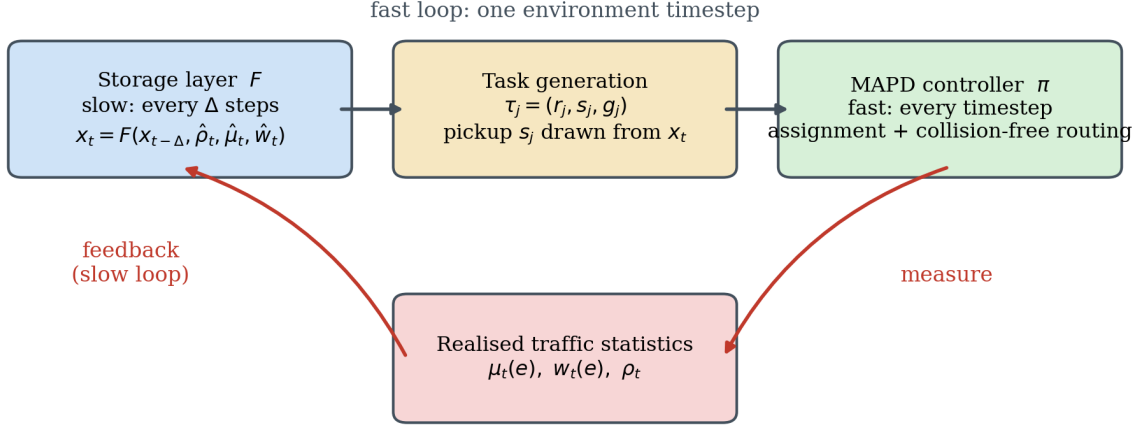


Fig. 5. The two-timescale loop. The fast path runs every timestep; the slow feedback path runs every Δ steps. Deleting the feedback path yields the fixed-storage baseline, so the experimental contrast of this thesis is a contrast between this diagram with and without one arrow.

Runs with $|\mathcal{C}_T| = 0$ are reported separately rather than assigned an arbitrary ratio. Because a completion-only statistic can improve while the system falls behind, the backlog $|\mathcal{Q}_T| + |\mathcal{B}_T|$ and the age distribution of unfinished tasks are always reported alongside (20).

Traffic distribution is measured separately from delay. Let $\mu_T(e)$ be the number of traversals of edge e over the evaluation window, $E_T^+ = \{e \in E : \mu_T(e) > 0\}$, and $p_T(e) = \mu_T(e) / \sum_{e' \in E} \mu_T(e')$. For $|E_T^+| > 1$ the normalised entropy is

$$H_T = -\frac{1}{\log |E_T^+|} \sum_{e \in E_T^+} p_T(e) \log p_T(e) \in [0, 1], \quad (22)$$

so $H_T = 1$ means perfectly uniform edge usage and $H_T \rightarrow 0$ means all traffic on one edge. Since entropy measures dispersion, the concentration index is its complement,

$$C_T = 1 - H_T. \quad (23)$$

The direction matters: adding $+H_T$ to a minimised cost would reward funnelling all agents down one aisle. Two caveats accompany (22). The normalisation depends on $|E_T^+|$, so runs using very different numbers of edges are not strictly comparable; and concentration is not congestion, because a concentrated flow at low load causes no delay. Congestion and concentration are therefore reported as separate outcomes. Figure 6 illustrates the two extremes.

The system-level criterion aggregates the minimisation-oriented measures,

$$\mathcal{J}(F, \pi) = q_{\text{svc}} \mathbb{E}[\bar{\zeta}_T] + q_{\text{mov}} \mathbb{E}[\bar{c}_T] + q_{\text{wait}} \mathbb{E}[W_T] + q_{\text{conc}} \mathbb{E}[C_T] + q_{\text{back}} \mathbb{E}[B_T], \quad (24)$$

with all $q_\bullet \geq 0$, $B_T = |\mathcal{Q}_T| + |\mathcal{B}_T|$, and W_T a waiting measure whose operational definition is given in Section ???. Throughput is deliberately excluded from (24): it is maximisation-oriented and, under a fixed arrival rate, largely redundant with $\bar{\zeta}_T$, so including it with a negative weight would double-count. It is reported separately.

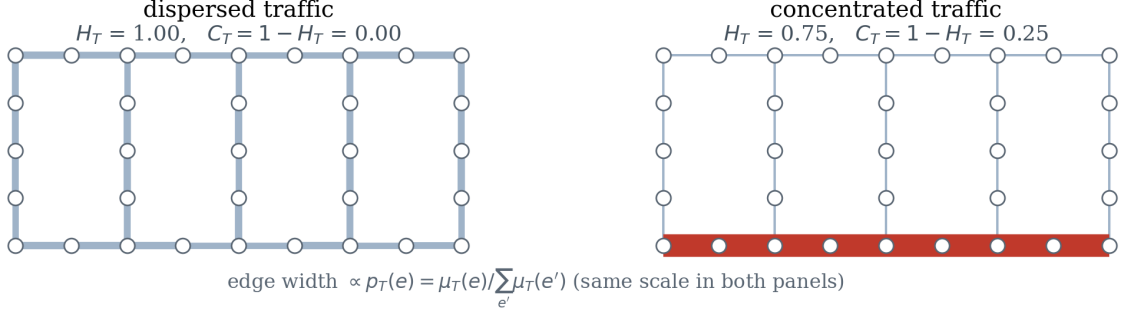


Fig. 6. Two edge-usage patterns on the same graph, drawn on a shared width scale, with the resulting H_T and C_T . The panels are schematic constructions that define the measure; measured values from experimental runs are reported in Section ??.

The formal decision problem is

$$(F^*, \pi^*) \in \arg \min_{F \in \mathcal{F}, \pi \in \Pi} \mathcal{J}(F, \pi) \quad \text{subject to (6), (3), (4), (5) and (10)}. \quad (25)$$

Equation (25) is a framing device. This thesis does not solve it globally; it estimates the effect of selected (F, π) combinations under a factorial design. Concretely, the empirical question is whether there exists an adaptive rule F such that

$$\mathcal{J}(F, \pi) < \mathcal{J}(F_{\text{fix}}, \pi) \quad \text{by a margin exceeding seed variance}, \quad (26)$$

and whether the size of that margin depends on the information regime of π given in Table I and on the load parameters m and λ_{task} . Section ?? realises every symbol appearing in (26).

J. Scope and assumptions

The model above is deliberately narrower than a physical warehouse, and the following are not represented. Agents are homogeneous, occupy exactly one vertex, and carry at most one task; there is no capacity, battery, or kinematic model, so turning and acceleration costs are absorbed into $c(e)$. Execution is deterministic: an accepted action always succeeds, so the transition kernel P is stochastic only through order arrivals. Human pickers and manual interventions are absent. Inventory relocation is abstract — (7) changes the placement used to generate future tasks, but the physical movement of stock is not simulated, and if $\chi = 0$ in (8) that movement is also free. Replenishment and depletion are modelled only if stated, and otherwise x_t is a placement template.

Each of these exclusions bounds the claims that the experiments can support. In particular, results about the value of congestion feedback are conditional on relocation being cheap, and results about decentralised control are conditional on deterministic execution.